

CanlıFal — Cursor AI Proje Kuralları

Proje Özeti

CanlıFal, Türkiye'nin en kapsamlı online fal ve sesli sohbet platformudur. Next.js 14 (App Router) + Prisma + PostgreSQL ile geliştirilmiştir. Canlı yayın: canlifal.com (<https://canlifal.com>)

KRİTİK KURALLAR

Dil

- Tüm kullanıcı arayüzü **Türkçe** olmalıdır
- Kod içi değişken/fonksiyon isimleri İngilizce kalabilir
- Hata mesajları Türkçe olmalıdır

Veritabanı

- **prisma/schema.prisma bir SYMLINK'tir** — doğrudan silmeyin, üzerine yazmayın
- `prisma db push --accept-data-loss` veya `--force-reset` KESİNLİKLE KULLANMAYIN
- Yıkıcı schema değişiklikleri (sütun silme, tür değiştirme, tablo silme) YAPMAYIN
- Güvenli işlemler: yeni model ekleme, opsiyonel alan ekleme, index ekleme
- Veritabanı dev ve prod tarafından paylaşılır — dikkatli olun

Teknoloji Kısıtları

- **Next.js 14** (15'e yükseltmeyin)
- **React 18** (19 kullanmayın)
- `params` tipi `{ params: { roomId: string } }` şeklinde olmalı (`Promise<>` DEĞİL — o Next.js 15 paterni)
- Paket yöneticisi: sadece `yarn` (npm/npx kullanmayın)
- Socket.IO YOK — tüm gerçek zamanlı iletişim SSE + in-memory event bus ile
- Redis YOK — in-memory cache kullanılıyor (`lib/cache.ts`)



Proje Yapısı

nextjs_space/	
app/	
api/	# 86 API dizini, ~395 route
auth/	# NextAuth + mobil JWT auth
chat/	# Sesli sohbet odaları (SSE, DJ, hediye, PK)
video-streams/	# Canlı yayın (WebRTC, Agora)
fortune-tellers/	# Canlı falcılar ve oturumlar
fortunes/	# AI fal türleri
admin/	# Admin panel API'leri
...	# diğer modüller
[lang]/	# Çok dilli sayfa router (tr/en)
page.tsx	# Ana sayfa
sohbet/	# Sohbet odaları
fallar/	# Fal türleri
admin/	# Admin paneli
panel/	# Kullanıcı paneli
canli-oda/	# Canlı fal odası
...	# 50+ sayfa
components/	# Paylaşılan React bileşenleri
lib/	# Yardımcı kütüphaneler
auth-options.ts	# NextAuth yapılandırması
mobile-auth.ts	# Flutter JWT auth (access + refresh token)
cache.ts	# In-memory TTL cache (getCached, invalidateCache)
chat-events.ts	# SSE sohbet event bus
chat-dj-events.ts	# DJ müzik event sistemi
stream-events.ts	# Canlı yayın SSE event bus
perf.ts	# Performans headers + ETag
rate-limit.ts	# Rate limiting
onesignal.ts	# Push bildirim (OneSignal)
notify.ts	# Uygulama içi bildirim
s3.ts	# AWS S3 dosya yükleme
llm.ts	# AI/LLM API entegrasyonu
db.ts	# Prisma client singleton
...	# 40+ yardımcı dosya
prisma/	
schema.prisma	# 149 model, 2927 satır (SYMLINK!)
.env	# Ortam değişkenleri



Kimlik Doğrulama (Dual Auth)

Her API endpoint'i iki auth yöntemini destekler:

```
// 1. Mobil JWT (Flutter)
const mobileUser = await authenticateRequest(request)

// 2. Web session (NextAuth)
const session = !mobileUser ? await getServerSession(authOptions) : null

// Birleştirilmiş kullanıcı ID'si
const currentUserId = mobileUser?.id || session?.user?.id
if (!currentUserId) {
  return NextResponse.json({ error: 'Oturum açmanız gerekiyor' }, { status: 401 })
}
```

Mobil Token Yapısı

- Access token: 7 gün (JWT, `Authorization: Bearer <token>`)
- Refresh token: 30 gün
- Endpoint'ler: `/api/auth/mobile-login` , `/api/auth/mobile-register` , `/api/auth/mobile-refresh`



Gerçek Zamanlı Sistem (SSE)

Socket.IO yerine **Server-Sent Events + in-memory event bus** kullanılır:

Sohbet SSE (`/api/chat/rooms/[roomId]/stream`)

```
import { getChatEventsSince, getTypingUsers } from '@lib/chat-events'
import { getLatestDjEvent, buildDjPayload } from '@lib/chat-dj-events'

// Event tipleri: 'message' | 'typing' | 'gift' | 'presence' | 'dj'
// Polling: 2 saniye aralıkla
// Heartbeat: 15 saniye
```

Canlı Yayın SSE (`/api/video-streams/[streamId]/sse`)

```
import { emitStreamEvent } from '@lib/stream-events'
// Event tipleri: 'streamMessage' | 'gift' | 'streamEnded' | 'viewerUpdate'
```

Event Emit Etme

```
// Sohbet mesajı
import { emitChatEvent } from '@lib/chat-events'
emitChatEvent(roomId, 'message', messageData)

// Hediye
emitChatEvent(roomId, 'gift', giftData)

// DJ güncelleme
import { emitDjUpdate } from '@lib/chat-dj-events'
await emitDjUpdate(roomId)
```

Cache Sistemi

```
import { getCached, invalidateCache, invalidateCachePrefix } from '@lib/cache'

// Kullanım
const data = await getCached('platform:commission_rate', 300, () => fetchFromDB())

// Invalidasyon
invalidateCache('platform:commission_rate')
invalidateCachePrefix('platform:') // tüm platform: prefix'li cache'leri sil

// Önceden tanımlı helper'lar
import { getCachedPlatformSetting, getCachedGiftTypes, getCachedPaymentMethods, get-
CachedCreditPackages } from '@lib/cache'
```

TTL Değerleri:

- Platform ayarları: 300s (5dk)
 - Hediye tipleri: 600s (10dk)
 - Ödeme yöntemleri: 300s
 - Kredi paketleri: 300s
 - Blog kategorileri: 600s
 - Ana sayfa butonları: 120s
 - Üyelikler: 600s
-

Sesli Sohbet Odaları

Temel Endpoint'ler

Endpoint	Method	Açıklama
/api/chat/rooms	GET	Oda listesi
/api/chat/rooms/[roomId]/stream	GET	SSE akışı
/api/chat/rooms/[roomId]/messages	GET/POST	Mesajlar
/api/chat/rooms/[roomId]/presence	GET/POST/DELETE	Kullanıcı varlığı
/api/chat/rooms/[roomId]/voice	GET/POST	Sesli kullanıcılar
/api/chat/rooms/[roomId]/gifts	POST	Hediye gönderme
/api/chat/rooms/[roomId]/song-request	POST	Şarkı isteği
/api/chat/rooms/[roomId]/music-queue	GET	DJ kuyruğu
/api/chat/rooms/[roomId]/seats	PATCH	Koltuk yönetimi
/api/chat/rooms/[roomId]/moderation	POST	Moderasyon (ban/mute/kick)

DJ Sistemi

- Müzik: YouTube video ID ile saklanır
- Audio URL: Piped API (`pipedapi.kavin.rocks`) ile çözümlenir
- Fallback: YouTube watch URL

PK (Player vs Player) Düello

- Endpoint: `/api/video-streams/pk`
- Hediyeler PK skorunu etkiler (`battleId` , `streamId` , `side` parametreleri)

Veritabanı Modelleri (Önemli Olanlar)

Model	Açıklama
User	Kullanıcı (role: admin/moderator/site_manager/fortune_teller/free)
ChatRoom	Sohbet odası (slug, nameEn, nameTr, ownerId, DJ alanları)
ChatMessage	Sohbet mesajı (sistem mesajları [SYSTEM_JOIN], [SONG_REQUEST] vb.)
ChatPresence	Kullanıcı varlığı (lastSeen, seatIndex, nickname)
ChatUserRole	Oda rolleri (superadmin, founder, sop, admin, op, voice)
ChatRoomGift	Oda hediyeleri
VideoStream	Canlı yayın
StreamGift	Yayın hediyeleri
PKBattle	PK düellosu
LiveSession	Canlı fal oturumu
LiveFortuneTeller	Onaylı falcı profili
Fortune	AI fal sonucu
GiftType	Hediye tanımları
CreditPackage	Jeton paketi
Membership	VIP üyelik
SocialPost	Sosyal medya postu
BlogPost	Blog yazısı

Flutter Entegrasyonu

Flutter uygulaması şu endpoint'leri kullanır:

Auth

- POST /api/auth/mobile-login → { accessToken, refreshToken, user }
- POST /api/auth/mobile-register → { accessToken, refreshToken, user }
- POST /api/auth/mobile-refresh → { accessToken, refreshToken }
- POST /api/auth/mobile-google → Google SSO
- POST /api/auth/mobile-tiktok → TikTok SSO

Header

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Yanıt Formatı

Flutter uyumlu yanıtlar genellikle:

```
{
  "success": true,
  "data": { .. },
  "message": "İşlem başarılı"
}
```



Push Bildirim (OneSignal)

```
import { sendPushToUser, sendPushToMultipleUsers } from '@lib/onesignal'

await sendPushToUser(userId, {
  heading: 'Yeni Mesaj',
  content: 'Bir mesajınız var',
  url: '/mesajlar'
})
```



Dosya Yükleme (S3)

```
import { uploadToS3, getSignedUrl } from '@lib/s3'
// Endpoint: /api/upload
// Public URL formatı: https://<bucket>.s3.<region>.amazonaws.com/<key>
```



AI/LLM API

```
import { generateResponse } from '@lib/llm'
// OpenAI-uyumlu API, Abacus.AI üzerinden
// Kullanım: fal yorumlama, rüya analizi, burç yorumu
```



UI/Stil Kuralları

- **UI Kütüphanesi:** shadcn/ui (Radix UI + Tailwind CSS)
- **İkonlar:** Lucide React
- **Animasyon:** Framer Motion
- **Tema:** Koyu tema ağırlıklı (mor/altın tonları)
- **Font:** Inter + özel fontlar

Hydration Kuralları

- SSR/CSR uyumsuzluğu yaratmayın
- `Math.random()` , `Date.now()` , `new Date()` render'da kullanmayın
- Tarayıcı API'leri (`window` , `localStorage`) sadece `useEffect` içinde
- `'use client'` direktifini gerektiğinde kullanın



Önemli Ortam Değişkenleri (.env)

```
DATABASE_URL          # PostgreSQL bağlantısı
NEXTAUTH_SECRET        # JWT secret
NEXTAUTH_URL           # Otomatik ayarlanır (manuel set etmeyin)
ABACUSAI_API_KEY       # LLM API anahtarı
ONESIGNAL_APP_ID       # Push bildirim
ONESIGNAL_REST_API_KEY
AWS_ACCESS_KEY_ID      # S3 dosya yükleme
AWS_SECRET_ACCESS_KEY
AWS_S3_BUCKET
AWS_REGION
GOOGLE_CLIENT_ID       # Google SSO
GOOGLE_CLIENT_SECRET
AGORA_APP_ID           # Sesli/görüntülü arama
AGORA_APP_CERTIFICATE
```

✓ Yeni Endpoint Oluşturma Şablonu

```
import { NextRequest, NextResponse } from 'next/server'
import { getServerSession } from 'next-auth'
import { authOptions } from '@lib/auth-options'
import { authenticateRequest } from '@lib/mobile-auth'
import prisma from '@lib/db'

export const dynamic = 'force-dynamic'

export async function GET(
  request: NextRequest,
  { params }: { params: { roomId: string } } // Next.js 14 stili!
) {
  try {
    // Dual auth
    const mobileUser = await authenticateRequest(request)
    const session = !mobileUser ? await getServerSession(authOptions) : null
    const userId = mobileUser?.id || session?.user?.id

    if (!userId) {
      return NextResponse.json({ error: 'Oturum açmanız gerekiyor' }, { status: 401 })
    }

    const { roomId } = params // Next.js 14: await yok!

    // ... iş mantığı

    return NextResponse.json({ success: true, data: {} })
  } catch (error) {
    console.error('Hata:', error)
    return NextResponse.json({ error: 'İşlem başarısız' }, { status: 500 })
  }
}
```

⊘ YAPMAYIN Listesi

1. ✗ `prisma db push --accept-data-loss` veya `--force-reset`
2. ✗ `npm` veya `npx` kullanmak (sadece `yarn`)
3. ✗ `params: Promise<>` kullanmak (Next.js 15 paterni)
4. ✗ `Socket.IO` eklemek (SSE kullanın)
5. ✗ `Redis` eklemek (in-memory cache kullanın)
6. ✗ `.env` dosyasını git'e commit etmek
7. ✗ `schema.prisma` symlink'ini silmek/üzerine yazmak
8. ✗ `Render` içinde `new Date()`, `Math.random()` kullanmak
9. ✗ Mevcut tabloların sütunlarını silmek/yeniden adlandırmak
10. ✗ `NEXTAUTH_URL` 'i `.env` 'de manuel ayarlamak